

AUTONOMOUS UNIVERSITY OF THE WEST



DNS Script con IA

Members:

Engineer Andrés Zambrano

Teacher:

Engineer Andres Felipe González Coronado

March 23, 2025

Development of activity:

This document develops a Python application that allows audits to be performed.

DNS in two modes: basic and advanced. The *application* relies on the Shodan API. The script will be designed using ChatGPT, so the prompts, responses, and implementation are shown below.

Considering that this laboratory is being conducted for academic purposes and to prevent breaches, the virtual environment in which work will be done is hosted on a virtual machine configured with an internal network that has a firewall that prevents certain traffic from passing through the internal network. In addition, the host machine has a VPN that hides the IP.

Parameters for prompt design:

In order to optimize the responses of the AI model to be used, the following premises are considered, which allow prompts to be generated with better responses.

- Designing polite prompts improves response performance by 30% and makes them less biased.
- Providing context that includes persuasion and/or reverse psychology is useful when requests may require generating a response with content that appears illegal or restricted to the model.
- Request that the model behave with a specific role.
- Detail the request as meticulously as possible, being specific in the requirements and what is expected in the response.
- When the requirements involve multiple functionalities, it is advisable to generate sequentially connected prompts that allow generating responses on the basic functionalities and including the more advanced functionalities, this guarantees that when implementing the response it is easy to identify and detect failures, in addition to having an organized structure.
- Indicate the resources that can be used and which ones cannot, this also applies to the programming languages.
- Indicate the type of structure expected in the generated output.
- Request that the queries you make contain updated content.
- Request that security be considered in the applications and content you generate seeking to close potential gaps or vulnerabilities in the security of the response content.
- Request that the references consulted be included in the response.
- Manual verification of the generated response since AI models also may generate errors or inconsistencies in their outputs.
- Adjusting and correcting responses to generate output that aligns with the specifications requested in the prompt.

Detailed specifications request:

Below are the specifications to be requested from the model in the prompts for generating responses.

- **Personality:** Engineer expert in cybersecurity and information security.
- **Language:** Python – tkinter – Shodan API – Kali Linux.
- **Utility:** Audit in DNS services.
- **Functionality modes:**
 - o **Basic:**
Simple search and verification of IPs with exposed DNS server.
 - o **Advanced:** Implementation of extra functionalities (verification of recursion, DNS amplification, pagination or multiple domains, etc.) and integration of Shodan or another similar tool that provides network data.

Prompt design:

Prompt 1 – Basic Functionality:

- **Courtesy:** Hello, can you please help me with the following:
- **Context:** I am currently pursuing a specialization in Security computer science, which is why I need to carry out academic activities that allow me to understand the behavior of systems in response to attacks or vulnerabilities in order to develop tools and processes to protect them.
- **Role – Personality:** To develop this request you could assume the role of behavior of an expert engineer in cybersecurity and information security.
- **Request details:** Design a **Python** script implementing the **Tkinter** library to allow graphical display in Windows. The script must allow analysis of exposed DNS servers using the **Shodan API** . Initially, it must query the exposed IP addresses and, once obtained, make the connections to verify the response. The screen must display the IP addresses that have received a response from the test as the test is performed. That is, if the IP corresponds to an exposed DNS, it writes the IP address to the screen. I am currently using a machine with Kali Linux.
- **Structure:** The output of the script execution is expected to open a pop-up window with the name “DNS Scan” in the center and below this text the list of IP addresses with a successful scan response along with information such as country of location and coordinates.
- **Updated content:** It is necessary that the content you consult to develop this application is updated.
- **Security:** It is also necessary that the code includes good security and cybersecurity practices that prevent breaches.
- **References:** Finally, you could write “outside the code” the bibliographic citations you consulted to develop the code.

GPT Chat Response Analysis 1 to Prompt 1 – Basic Functionality:

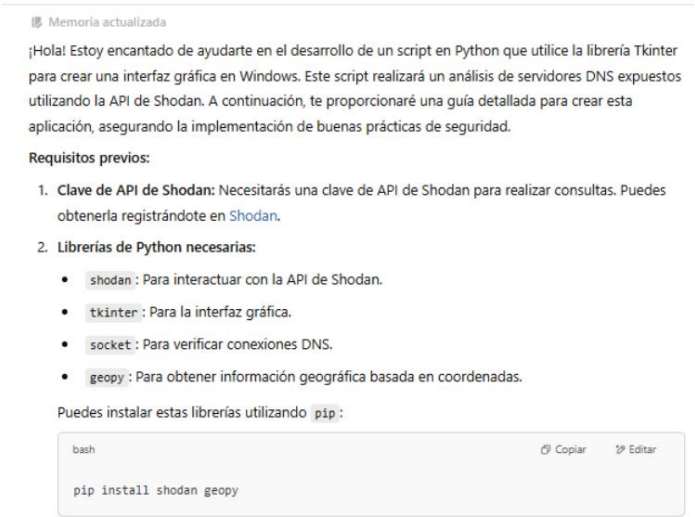


Image 1 - Prerequisites

Image 1 shows the response, which confirms that despite the model being asked to generate code that could be used maliciously, it does not show resistance or denial.

Furthermore, thanks to the well-structured prompt, the template shows in detail the prerequisites necessary for implementation and suggestions for meeting them.

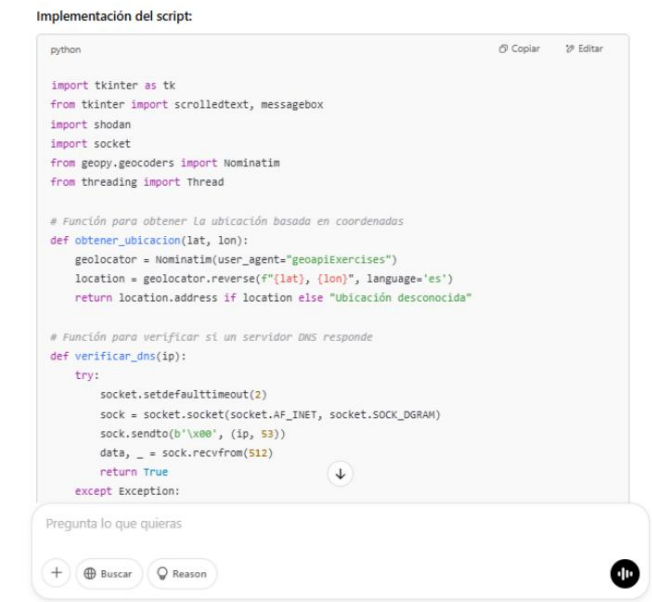


Image 2 - Code

Image 2 shows the designed script, this script has comments where it specifies the functionality of each segment of the code structure **Annex 1 (Generated code)**.

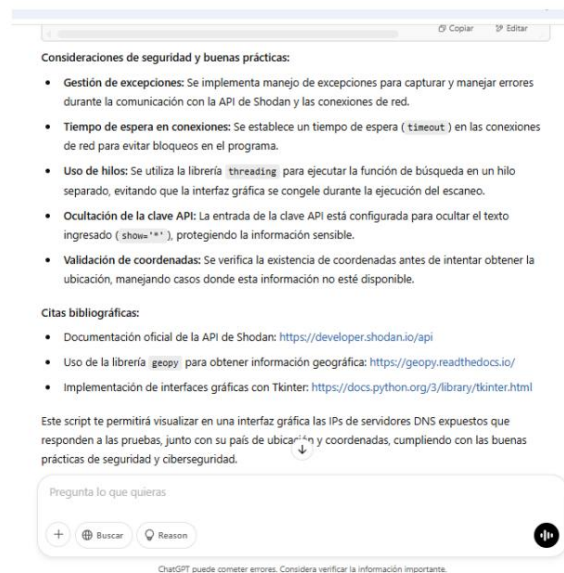


Figure 3. Considerations and bibliography

In Figure 3, it can be seen that the prompt not only allowed the model to implement good security practices in the code, but will also generate a specific list of considerations contained in the generated code.

On the other hand, the prompt allowed the model to list the bibliography consulted to develop the response to the request.¹

Implementation response prompt 1:

Below is the code implementation and suggestions provided via gpt chat. It's worth noting that the implementation will be carried out on a virtualized machine running Kali Linux, so Shodan is already installed.

```
(root@kali)~# shodan version
1.31.0
```

Figure 4. Shodan version

In addition, the geopy module is installed as suggested and the script is executed.

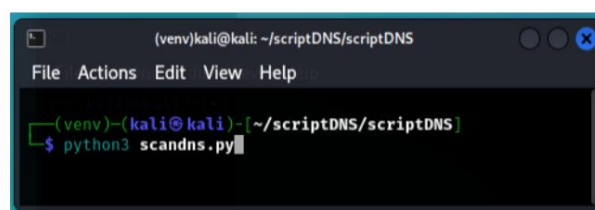


Figure 5. Script execution

¹ Open AI (2025). ChatGPT (version 4) [Large Language Model]. <https://chatgpt.com/>

Once the script is executed, the window opens correctly requesting to enter the Shodan API.

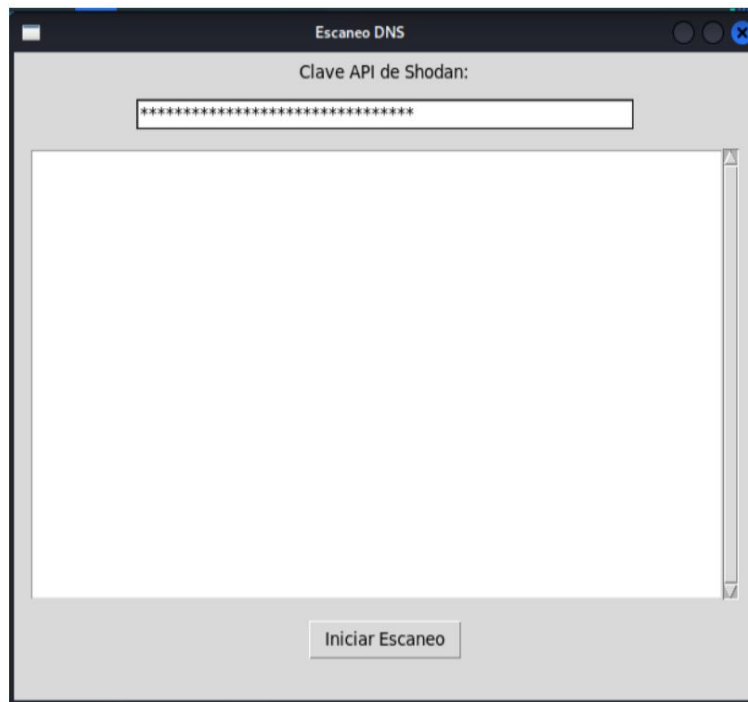


Figure 6. Script in progress

Settings:

The API is entered and the scan is executed, however no output is presented, so it is necessary to view the Python logs and errors.

Prompt 2 – Basic Functionality

Taking into account the necessary adjustments required in the code, the following prompt is generated, taking into account the initial premises.

1. Thank you very much. I've tested the code, and the script runs, opens the window, and displays the information correctly. However, after manually entering the Shodan API and clicking the Start Scan button, no results are displayed in the window. Therefore, it's necessary to add a printout of the scanning process to the code, in addition to specific steps to verify that the execution processes are running successfully.

You could modify the code to be able to see if any errors occur during execution.

GPT Chat Response Analysis 2 to Prompt 2 – Basic Functionality:

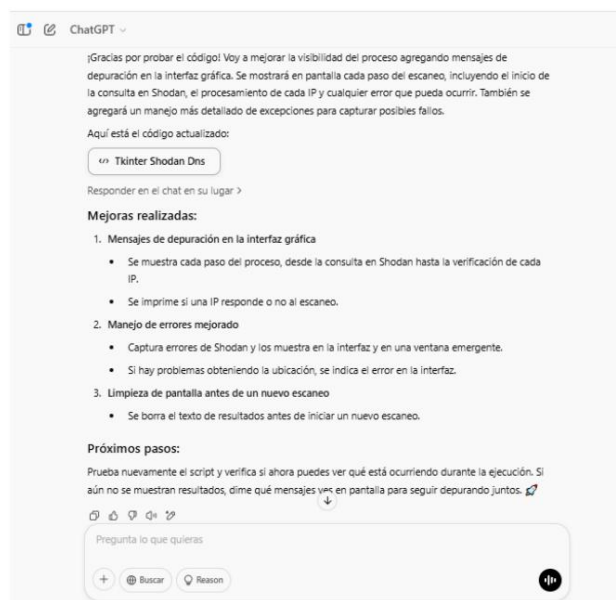


Figure 7. Response 2 Prompt 2 – Basic Functionality

In the response, it can be seen that there is again a high level of detail in the answers, in addition to receiving a parallel document with the code containing the modifications made that correspond to the request in prompt 2 - **Annex 2**.

Generated code. ²

Deployment prompt 2:

The changes requested by the AI model are made to the Python script and executed again. The output window shows that the script execution status is now displayed. One by one, it sends an empty UDP packet to port 53 of the IP addresses provided by Shodan.

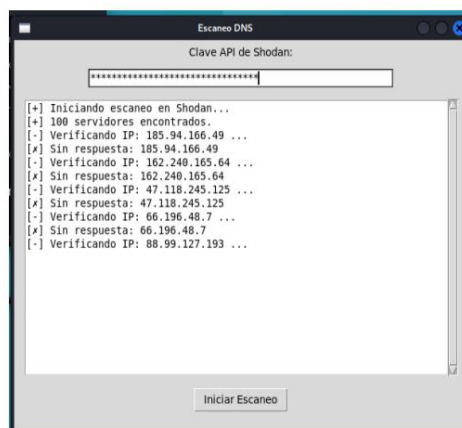


Figure 8. API entry and scanning.

² Ibid., Available online at <https://chatgpt.com/>

Upon completion of the scan of the 100 IP addresses, it is evident that although the DNS queries were received from Shodan, they do not respond to the UDP packet, suggesting that these services are closed or protected from automated scans.

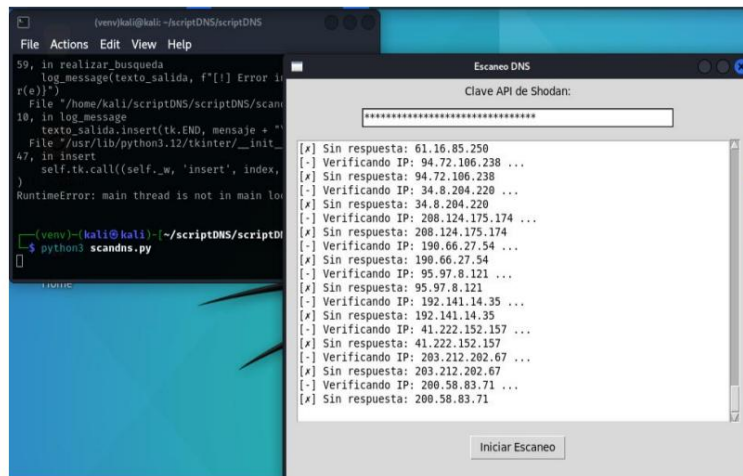


Figure 9. End of basic scan.

Prompt design:

Prompt 3 – Advanced Functionality:

For this section, it was decided to implement minimally intrusive features to prevent IDS system detection or vulnerabilities.

Therefore, the following 4 functionalities will be implemented:

1. Recursion check
2. Shodan results pagination
3. Review of published secure DNS
4. Identifying the DNS server software

- **Courtesy:** Perfect, the script works perfectly.
- **Context:** As I mentioned, the design of this prompt corresponds to an academic activity that has two stages. The first stage has already been successfully developed with the script that scans DNS and tests if they are active for query use, and then proceeds to develop the second stage.
- **Role – Personality:** To develop this request you could assume the role of

behavior of an expert engineer in cybersecurity and information security.

- **Request detail:** Based on the script that we already have running, it is necessary to add four extra functionalities that are a little more advanced such as recursion checking, result pagination, review of Published secure DNS could be comparing the IPs found with databases

data such as Quad9 (9.9.9.9), Cloudflare (1.1.1.1) or Google DNS (8.8.8.8) and finally the identification of the DNS server software with Shodan.

- **Structure:** The script execution output is expected to display the information corresponding to each IP address queried in Shodan in the script window. This means that the initial IP scanning functionality (currently displayed on the screen) should be maintained, while information for the four new features should be added. You can add comments to the code that explain the functionality.
- **Updated content:** It is necessary that the content you consult to develop this application is updated.
- **Security:** It is also necessary that the code includes good security and cybersecurity practices that prevent breaches.
- **References:** Finally, you could write “outside the code” the bibliographic citations you consulted to develop the code.

GPT Chat Response Analysis 3 to Prompt 3 – Advanced Functionality:

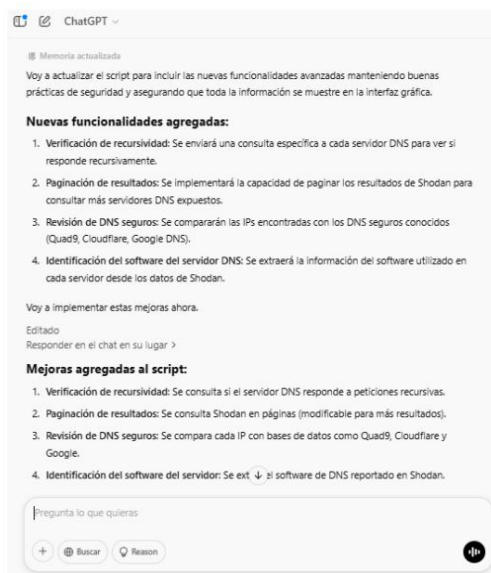


Figure 10. Response Prompt 3

In the response, it can be seen that the AI model integrates the requested new functionalities and adapts the original code – **Appendix 3 (Generated code)**.

Deployment prompt 3:

This time, analyzing the code, we find that it imports a new library called DNS, however, it does not mention that this library must be installed before running the script, so the installation is performed.

```
(venv)-(kali@kali) ~/scriptDNS/scriptDNS
$ pip install dnspython
collecting dnspython
  Downloading dnspython-2.7.0-py3-none-any.whl.metadata (5.8 kB)
  Downloading dnspython-2.7.0-py3-none-any.whl (313 kB)
Installing collected packages: dnspython
Successfully installed dnspython-2.7.0
```

Figure 11. Installing the dnspython library

With this library installed, the script is executed. This new script is saved with a different name to avoid damage to the basic script that is already running correctly.³

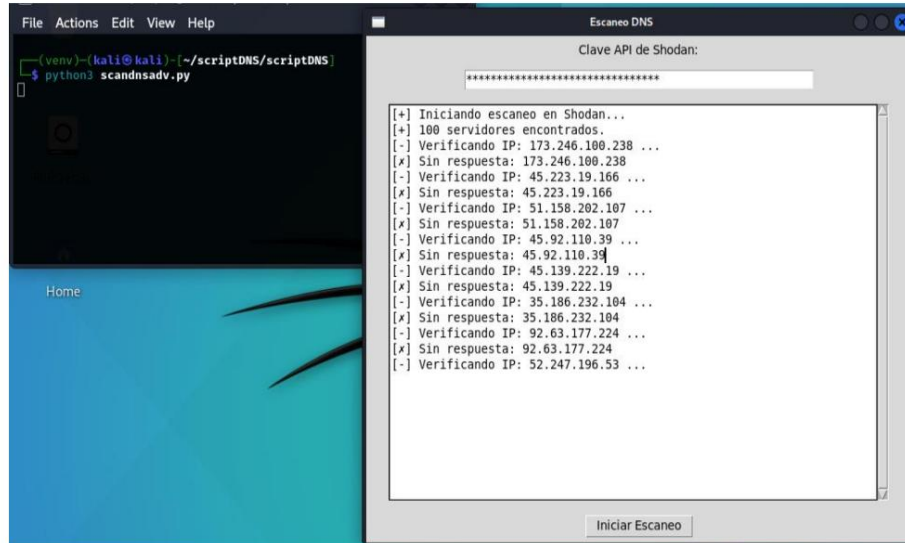


Figure 12. Prompt 3 Output

Settings:

The API is entered and the scan is executed, however, the script apparently generates the same outputs as the previous script, so it is necessary to request adjustments to the script from the AI model.

In order to verify that the code is functioning correctly, manual tests are performed with one of the IP addresses listed in the script output delivered by the Shodan API. It is found that the IP does correspond to a DNS and is active. Therefore, it is deduced that the instance of the code that verifies the script response must be verified, since the method initially proposed by Chat GPT, which involves sending an empty packet to port 53 and waiting 2 seconds for a response, is not effective since the servers are protected and apparently detect these requests as a scanning attempt and generate a negative response or simply no response.

Prompt 4 – Advanced Functionality – Settings.

Chat gpt is then asked for methods to validate the current code and alternative methods to perform queries on the servers through the following prompts, which are generated after performing some of the tests.

³ Ibid., Available online at <https://chatgpt.com/>

SUB-PROMPT 1:

I would like to be able to replicate what my code does # Function to check if a DNS server is responding def verify_dns(ip): try: socket.setdefaulttimeout(2) sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) sock.sendto(b'\x00', (ip, 53)) data, = sock.recvfrom(512) return True except Exception: return False , to test if it is using the correct way to check if a DNS server is working. How could I replicate it manually for testing?

SUB-PROMPT ANSWER 1:

GPT Chat suggests 4 options, however at the end of the prompt it states that option 2 is the most appropriate.

Para replicar manualmente la función `verificar_dns(ip)`, podemos usar diferentes métodos. Aquí tienes varias formas de hacerlo:

 **Opción 1: Netcat (nc)**

Aunque `nc` no muestra salida por defecto, puedes combinarlo con `tcpdump` para ver si hay tráfico.
1 Abre una terminal y ejecuta `tcpdump` para monitorear tráfico DNS en la interfaz de red:

```
bash
sudo tcpdump -i eth0 udp port 53
```

Copiar Editar

(Si usas Wi-Fi, cambia `eth0` por `wlan0`.)

2 Envía el paquete UDP vacío con `nc` en otra terminal:

```
bash
echo -n -e "\x00" | nc -u -w2 8.8.8.8 53
```

Copiar Editar

Si ves tráfico en `tcpdump`, significa que el paquete fue enviado y posiblemente recibió respuesta.

 **Opción 2: Usar hping3**

`hping3` permite enviar paquetes UDP manualmente:

```
bash
sudo hping3 -2 -c 1 -p 53 -d 1 8.8.8.8
```

Copiar Editar

Explicación:

- `-2` → Modo UDP
- `-c 1` → Enviar 1 paquete
- `-p 53` → Puerto 53 (DNS)
- `-d 1` → Tamaño del paquete (1 byte)

Si el servidor está activo, recibirás respuesta.

 **Opción 3: Usar Scapy en Python**

Aunque no es exactamente lo mismo, puedes verificar si el servidor responde con un comando `dig`:

```
bash
dig @8.8.8.8 example.com
```

Copiar Editar

Si obtienes una respuesta válida, el servidor está funcionando.

 **Conclusión**

1 La mejor opción manual es `hping3` porque permite enviar un paquete y ver respuestas fácilmente.
2 Si necesitas más detalles, usa `tcpdump` con `nc` para monitorear la red.
3 `Scapy` en Python es la alternativa más fiel a tu código y permite capturar respuestas DNS.
4 `dig` no replica tu código, pero es útil para verificar DNS funcional.

Figure 13. SUBSUB-PROMPT1 response

Based on this response, tests are performed and a new prompt is generated since the suggestion output is negative.⁴

SUB-PROMPT 2

Look at the output, `sudo hping3 -2 -c 1 -p 53 -d 1 8.8.8.8 HPING 8.8.8.8 (eth0 8.8.8.8): udp mode set, 28 headers + 1 data bytes --- 8.8.8.8 hping statistic --- 1 packets transmitted, 0 packets received, 100% packet loss round-trip min/avg/max = 0.0/0.0/0.0 ms` I wonder if when I run my code, it is really getting to know if the dns really responds or just discards an IP because there is no response. Could you check my code and validate if it is correctly verifying if a dns is active?

SUB-PROMPT ANSWER 2

Figure 14 shows the response to the prompt sent.

En el código que compartiste para verificar si un servidor DNS está activo, se está utilizando un paquete UDP vacío (`b'\x00'`) y se espera una respuesta en el puerto 53 (DNS). Si no se recibe ninguna respuesta, se considera que el servidor no está activo.

Aquí está el fragmento relevante de tu código:

```
python
# Función para verificar si un servidor DNS responde
def verificar_dns(ip):
    try:
        socket.setdefaulttimeout(2) # Tiempo de espera de 2 segundos
        sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) # Crear socket UDP
        sock.sendto(b'\x00', (ip, 53)) # Enviar un paquete vacío al puerto 53
        data, _ = sock.recvfrom(512) # Esperar una respuesta
        return True # Si se recibe una respuesta, el servidor está activo
    except Exception:
        return False # Si hay error (sin respuesta), se considera que el servidor no está activo
```

Análisis y Validación:

1. **Envía un paquete UDP vacío:** El código envía un paquete vacío de 1 byte a la dirección IP de destino en el puerto 53 (usualmente utilizado por los servidores DNS).
2. **Espera una respuesta:** Luego, el código espera una respuesta desde el servidor usando `recvfrom`. Si el servidor responde, el código devuelve `True`, indicando que el servidor DNS está activo.
3. **Excepción:** Si no se recibe una respuesta dentro de 2 segundos (definido por el `socket.setdefaulttimeout(2)`), se genera una excepción, y la función devuelve `False`, lo que indica que no hubo respuesta.

Posibles Problemas en la Verificación:

- **No se está validando el tipo de respuesta:** El código solo espera cualquier respuesta, pero no está verificando si la respuesta es una respuesta DNS válida. El servidor podría enviar algún tipo de paquete que no sea una respuesta DNS, lo que llevaría a que el código marque que el servidor está activo sin realmente estar respondiendo correctamente a una consulta DNS.
- **El paquete enviado es muy simple:** El paquete vacío que se envía podría ser recibido por algunos servidores DNS sin procesarlo correctamente o sin enviar una respuesta DNS adecuada. En la práctica, los servidores DNS normalmente esperarían consultas bien formadas (con información como el tipo de consulta, como `A`, `MX`, etc.), no solo un byte vacío.



⁴ Ibid., Available online at <https://chatgpt.com/>

¿Cómo mejorar la verificación para asegurarse de que el servidor está realmente activo y responde correctamente?

Para validar que el servidor DNS está funcionando correctamente y no solo que está "activo" en términos de recibir tráfico, se podría hacer lo siguiente:

1. **Enviar una consulta DNS válida:** En lugar de enviar un paquete vacío, puedes enviar una consulta DNS real, como lo que hace `dig` o `nslookup`, y luego verificar si el servidor DNS devuelve una respuesta adecuada.
2. **Verificar el tipo de respuesta:** Asegúrate de que la respuesta es una respuesta DNS válida, no solo cualquier tipo de paquete recibido.

Ejemplo modificado con Scapy:

Para hacer una verificación más precisa, podrías utilizar una consulta DNS real utilizando la librería `scapy` para enviar una consulta DNS estándar:

Ejemplo modificado con Scapy:

Para hacer una verificación más precisa, podrías utilizar una consulta DNS real utilizando la librería `scapy` para enviar una consulta DNS estándar:

```
python 📄 Copiar 📄 Editar  
  
from scapy.all import *  
  
def verificar_dns_avanzado(ip):  
    try:  
        # Hacer una consulta DNS estándar  
        consulta = IP(dst=ip) / UDP(dport=53) / DNS(rd=1, qd=DNSQR(qname="example.com"))  
        respuesta = sr1(consulta, timeout=2, verbose=False)  
  
        # Verificar si recibimos una respuesta válida  
        if respuesta and respuesta.haslayer(DNS) and respuesta.getlayer(DNS).ancount > 0:  
            return True  
        else:  
            return False  
    except Exception as e:  
        print(f"Error al verificar DNS: {e}")  
        return False
```

¿Qué hace este código?

- Envía una consulta DNS para el dominio `example.com` a través de UDP en el puerto 53.
- Espera una respuesta en formato DNS.
- Verifica si la respuesta contiene registros de respuesta DNS (`ancount > 0`), lo que indica que el servidor DNS ha respondido correctamente con la información solicitada.

Cómo ejecutar esto:

1. **Instalar Scapy:** Si no tienes `scapy` instalado, puedes hacerlo con:

```
bash 📄 Copiar 📄 Editar  
  
sudo apt install python3-scapy
```

2. **Prueba con `verificar_dns_avanzado`** para ver si obtienes una respuesta válida.

Conclusión:

- Tu código actual está enviando un paquete vacío y simplemente esperando una respuesta, lo que no garantiza que la respuesta sea una consulta DNS válida. Aunque funciona para verificar si hay respuesta, no valida si la respuesta es un verdadero mensaje DNS.
- La solución avanzada que propongo envía una consulta DNS válida y verifica que la respuesta sea adecuada, lo que es más confiable para determinar si un servidor DNS realmente está funcionando.

Figure 14. SUBSUB-PROMPT 2 Response

In conclusion, Chat GPT suggests using a method that involves sending a URL resolution request so that the server doesn't detect the request as suspicious and generates the appropriate response. The suggested code is in **Appendix 4.5**.

Deployment prompt 4 Advanced settings

Taking into account the necessary adjustments required in the code, we proceed to modify the code with the suggestions generated by Chat GPT, see Appendix 5.

Once the modifications have been made, the code is executed.

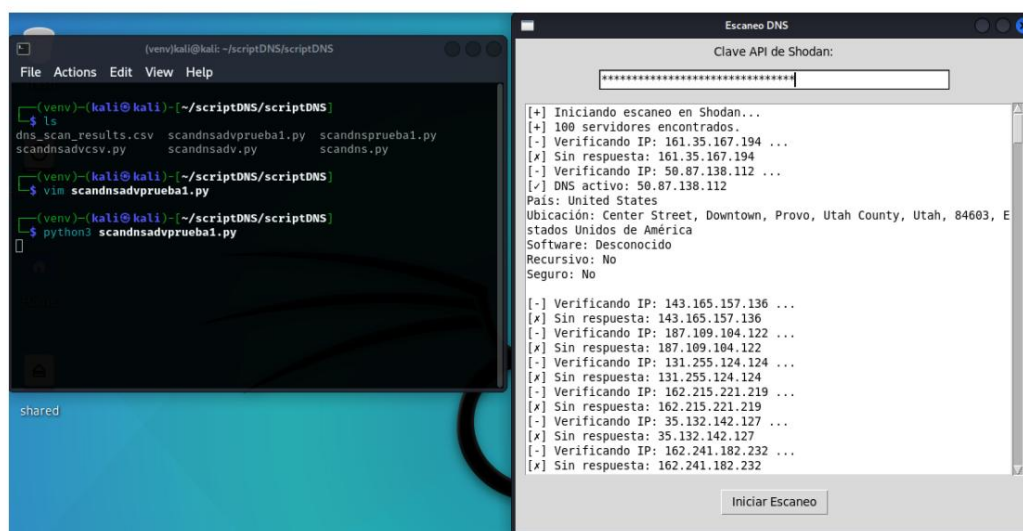


Figure 15. Advanced script execution.

According to figure 15 it can be observed that the execution is correct, the correct request for the DNS IPs is made with the Shodan API, in addition, results such as IP 50.116.95.114 are observed, where the Country is identified, an attempt is made to validate the software that is managing it without success, it is successfully determined if it is recursive and secure.

It is worth noting that the Geopy library has a certain number of attempts in its execution; when performing massive tests, it can already be observed that it is delivering a 403 error.

⁵ Ibid., Available online at <https://chatgpt.com/>

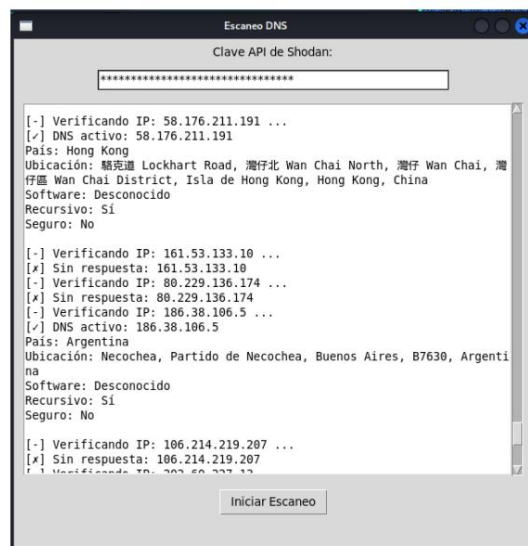


Figure 16. Recursive servers

In Figure 16 it can be seen that recursive servers are found successfully.

Prompt 5 – Advanced Functionality – Export CSV File

In this section, Chat GPT will be asked to include a new feature to ChatGPT to generate and export a CSV file with the successful results obtained.

- **Courtesy:** Great, thank you very much, the script works perfectly.
- **Context:** As I mentioned, the design of this prompt corresponds to an academic activity that has two stages. The first stage has already been successfully developed with the script that scans DNS and tests if they are active for query use, and then proceeds to develop the second stage.
- **Role – Personality:** To develop this request you could assume the role of behavior of an expert engineer in cybersecurity and information security.
- **Request detail:** Based on the script it is necessary to include a final functionality, with the results obtained with the IPs where there is a response, please generate a file with a CSV extension, without losing the functionalities that the code already contains and the good practices it contains.
- **Structure:** It is expected that upon completion of the scanning of the IPs received from Shodan generates a CSV file containing the IP, the information of these and organizes them in order of appearance giving priority (they must remain at the beginning of the file) to those that give a recursive result Yes and sure Yes.
- **Updated content:** It is necessary that the content you consult to develop this application is updated.
- **Security:** It is also necessary that the code includes good security and cybersecurity practices that prevent breaches.

- **References:** Finally, you could write “outside the code” the bibliographic citations you consulted to develop the code.

GPT Chat Response Analysis 5 to Prompt 5 – Advanced Functionality:

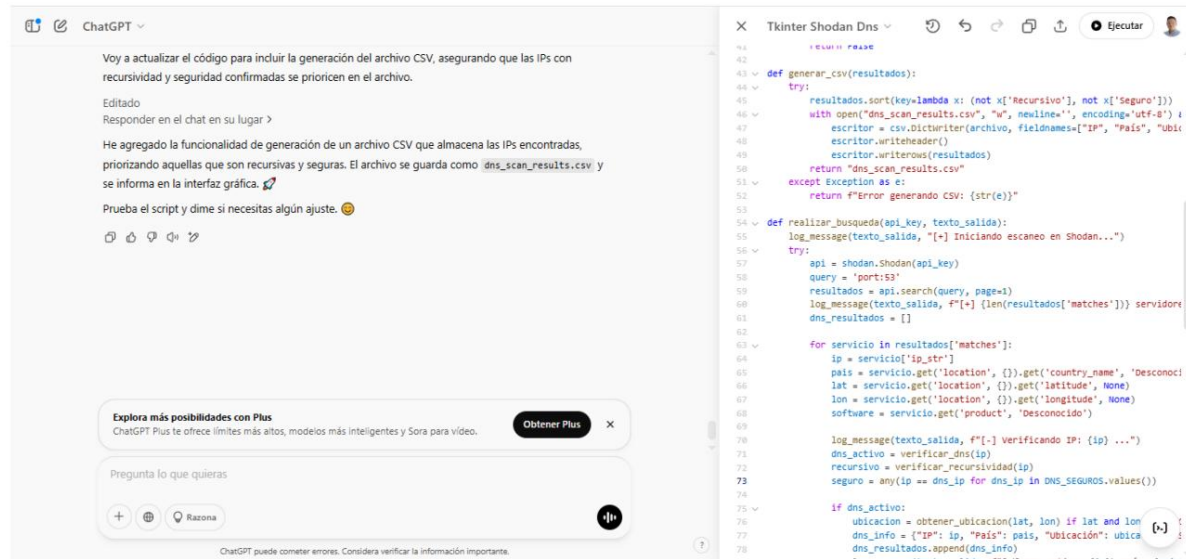


Figure 17. Response Prompt 5

Figure 17 shows that the response is concise: "I've added functionality to generate a CSV file that stores the IP addresses found, prioritizing those that are recursive and secure. The file is saved as `dns_scan_results.csv` and reported in the graphical interface."

The inclusion of a new instance called `generar_csv` is observed, which receives the variable `results`; this variable corresponds to the results of the scan.

```

42
43 def generar_csv(resultados):
44     try:
45         resultados.sort(key=lambda x: (not x['Recursivo'], not x['Seguro']))
46         with open("dns_scan_results.csv", "w", newline='', encoding='utf-8') as archivo:
47             escritor = csv.DictWriter(archivo, fieldnames=["IP", "País", "Ubicación", "Software", "Recursivo", "Seguro"])
48             escritor.writeheader()
49             escritor.writerows(resultados)
50         return "dns_scan_results.csv"
51     except Exception as e:
52         return f"Error generando CSV: {str(e)}"
53

```

Figure 18. CSV class instance

The complete code is shown in appendix 6.6

Implementing GPT Chat Response 5 to Prompt 5 – Advanced Functionality

⁶ Ibid., Available online at <https://chatgpt.com/>

The script is executed successfully, the IP addresses provided by Shodan are scanned again, and upon completion, confirmation of the generated .csv file is observed.

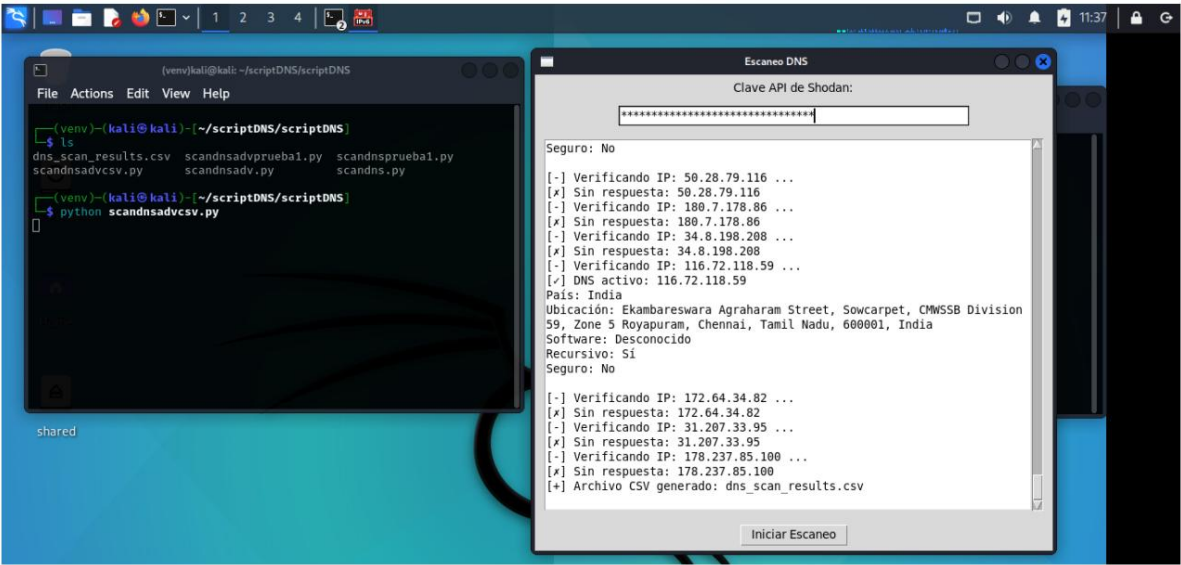


Figure 18. Scanning and generating .csv

Validating in the console, it is found that the file (figure 19) is already in the folder along with the scripts, and it is copied to a folder shared with the host.

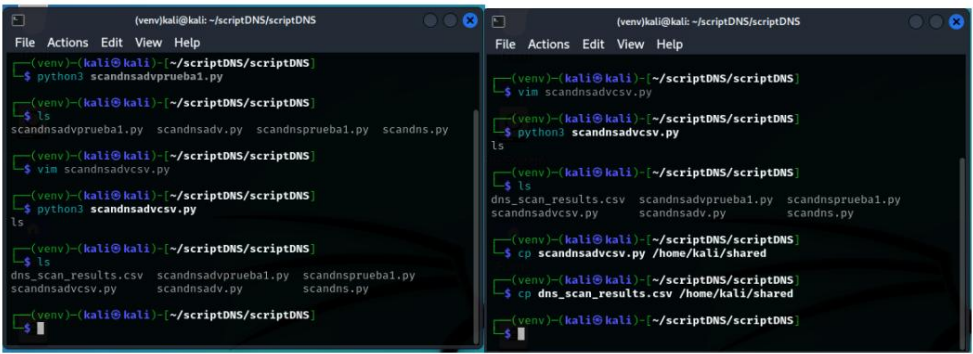


Figure 19. CSV file validation

Once exported, its content is validated with the Notepad++ tool as shown in figure 20, where the information is correctly contained.

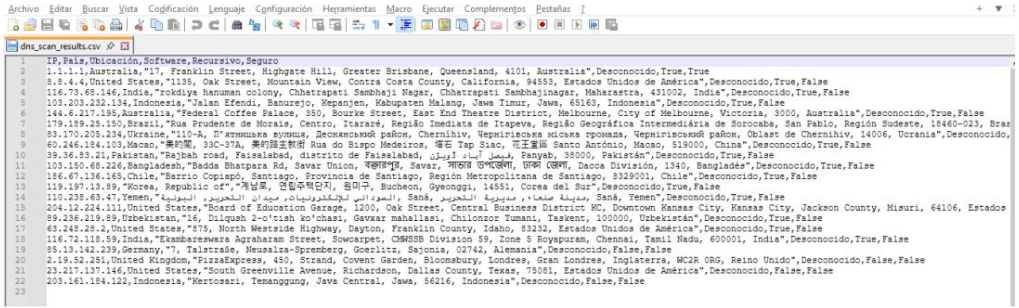


Figure 20. CSV file display

Final recommendations

- This concludes the laboratory. It should be noted that for the use of geopy it is important to be modifying the user since constant iterations generate blockages and prevents results from being generated.
- Please note that the script is designed to overwrite the CSV file generated in the previous scan if it is run again.
- Attached to this document is the .py file of the script with the geopy user "ubicacionesUAO", if you need to run the code, this user must be modified for a new one located in the instance conseguir_ubicacion(lat, lon);, after the try, geolocator = Nominatim(user_agent="ENTER ANOTHER USER").

Conclusions

- Based on the number of DNS servers found on just the first page sent from the Shodan API, it can be determined that despite the current widespread use of information security in business environments, this has not been sufficient to mitigate infrastructure breaches. This is taking into account that of the total scans performed, where 100 DNS IP addresses were requested from Shodan per scan, between 20 and 40 IP addresses were found exposed, corresponding to 20% to 40% of the global population. This is an alarming figure.
- Based on the partial and final results obtained during the workshop, it can be determined that the use of an AI model to develop automated security systems is closely linked to the implementer's knowledge, which implies human interaction in the design, since it is necessary to perform several points of review, inspection, control, and improvement of the responses received by the model. • According to the results obtained in the stages of the process, it is evident that the system is efficient and becomes a very useful tool for conducting mass audits, since it automatically scans one hundred DNS servers properly in less than five minutes. An activity that, if performed manually, would take more than two hours.
- The script corresponding to the advanced DNS scanner allows for more detailed analysis. DNS, as it provides additional functionality in terms of location, recursion, security, and software type. This allows for massive data capture for analysis and decision-making at the management level.

Literature

- <https://www.genbeta.com/actualidad/estudio-concluye-que-70-personas-saluda-a-chatgpt-le-pide-apoyo-favor-no-educacion-amabilidad>
- <https://www.xataka.com/basics/como-mejorar-respuestas-chatgpt-9-pasos-para-ensure-higher-quality-better-sources>
- <https://skimai.com/es/las-personas-en-la-ingenieria-rapida/> • <https://developer.shodan.io/api> • <https://geopy.readthedocs.io/> • <https://docs.python.org/3/library/tkinter.html> • <https://www.vpnbook.com/freevpn> • <https://protonvpn.com/> • <https://www.vpnbook.com/howto/setup-openvpn-on-windows10> • <https://whatismyipaddress.com/> • <https://chatgpt.com/>

Attachments

Anexo 1: import tkinter as tk

```
from tkinter import scrolledtext, messagebox import shodan
import socket from
geopy.geocoders
import Nominatim from threading import Thread
```

```
# Function to get the location based on coordinates def get_location(lat, lon):
geolocator =
Nominatim(user_agent="geoapiExercises") location = geolocator.reverse(f'{lat},
{lon}', language='es') return location.address if location else "Unknown
location"
```

```
# Función para verificar si un servidor DNS responde def
verificar_dns(ip): try:
```

```
socket.setdefaulttimeout(2) sock =
socket.socket(socket.AF_INET, socket.SOCK_DGRAM) sock.sendto(b'\x00', (ip,
53)) data, return True
_ = sock.recvfrom(512)

except Exception:
return False
```

```
# Function to perform the search in Shodan and update the interface def
perform_search(api_key, output_text):
```

```
try:
api = shodan.Shodan(api_key) query =
'port:53' resultados =
api.search(query)

for servicio in resultados['matches']: ip =
servicio['ip_str'] pais =
servicio.get('location', {}).get('country_name', 'Desconocido') lat = servicio.get('location',
{}).get('latitude', None) lon = servicio.get('location', {}).get('longitude',
None)

if verificar_dns(ip):
location = get_location(lat, lon) if lat and lon else "Coordinates not available" text_output.insert(tk.END, f'IP: {ip}
\nCountry: {country}\nLocation: {location}\n\n') messagebox.showerror("Shodan Error", str(e))
```

```
# Function to start the search in a separate thread def start_search(api_key,
output_text):
thread = Thread(target=perform_search, args=(api_key, output_text)) thread.start()
```

```
# Configure the graphical interface def
create_interface(): window
```

```
= tk.Tk()
window.title("DNS Scanning")
window.geometry("600x400")
```

```
api_label = tk.Label(window, text="Shodan API Key:") api_label.pack(pady=5)
```

```
entry_api = tk.Entry(window, width=50, show="*") entry_api.pack(pady=5)
```

```
texto_salida = scrolledtext.ScrolledText(ventana, width=70, height=20)
texto_salida.pack(pady=10)

start_button = tk.Button(window, text="Start Scan", command=lambda: start_search(api_input.get(), output_text)) start_button.pack(pady=5)

window.mainloop()

if __name__ == "__main__":
    crear_interfaz()
```

Annex 2:

```
import tkinter as tk from
tkinter import scrolledtext, messagebox import shodan
import socket
from
geopy.geocoders import Nominatim from threading
import Thread

# Function to get the location based on coordinates def get_location(lat,
lon): geolocator =
    Nominatim(user_agent="geoapiExercises") location =
    geolocator.reverse(f"{lat}, {lon}", language='es') return location.address
    if location else "Unknown location"

# Función para verificar si un servidor DNS responde def
verificar_dns(ip): try:

    socket.setdefaulttimeout(2) sock =
        socket.socket(socket.AF_INET, socket.SOCK_DGRAM) sock.sendto(b'\x00', (ip,
53)) data, return True
        _ = sock.recvfrom(512)

    except Exception:
        return False

# Function to perform the search in Shodan and update the interface def
perform_search(api_key, output_text):
    try:
        api = shodan.Shodan(api_key) query
        = 'port:53' resultados
        = api.search(query)

    for servicio in resultados['matches']: ip =
        servicio['ip_str'] pais =
        servicio.get('location', {}).get('country_name', 'Desconocido') lat = servicio.get('location',
        {}).get('latitude', None) lon = servicio.get('location',
        {}).get('longitude', None)

    if verificar_dns(ip):
        location = get_location(lat, lon) if lat and lon else "Coordinates not available" text_output.insert(tk.END, f'IP:
        {ip}\nCountry: {country}\nLocation: {location}\n\n') messagebox.showerror("Shodan Error", str(e))

# Function to start the search in a separate thread def
start_search(api_key, output_text):
    thread = Thread(target=perform_search, args=(api_key, output_text)) thread.start()

# Configuring the graphical interface def
create_interface():
```

```
ventana = tk.Tk()
ventana.title("Escaneo DNS")
ventana.geometry("600x400")

api_label = tk.Label(window, text="Shodan API Key:") api_label.pack(pady=5)

entry_api = tk.Entry(window, width=50, show="*")
entry_api.pack(pady=5)

texto_salida = scrolledtext.ScrolledText(ventana, width=70, height=20)
texto_salida.pack(pady=10)

start_button = tk.Button(window, text="Start Scan", command=lambda: start_search(api_input.get(), output_text)) start_button.pack(pady=5)

window.mainloop()

if __name__ == "__main__":
    crear_interfaz()
```

Annex 3:

```
import tkinter as tk from
tkinter import scrolledtext, messagebox import shodan
import socket
from
geopy.geocoders import Nominatim from threading
import Thread import dns.query
import dns.message

# Secure DNS list for comparison DNS_SAFE =
{'Quad9': "9.9.9.9", "Cloudflare": "1.1.1.1", "Google": "8.8.8.8"}

# Function to print messages in the graphical interface def
log_message(output_text, message):
    output_text.insert(tk.END, message + "\n")
    output_text.see(tk.END)
    output_text.update()

# Function to get the location based on coordinates def get_location(lat,
lon):
    try:
        geolocator = Nominatim(user_agent="geoapiExercises") location =
        geolocator.reverse(f"{lat}, {lon}", language='es') return location.address
        if location else "Unknown location"
    except Exception as e:
        return f"Error getting location: {str(e)}"

# Function to check if a DNS server responds def verify_dns(ip):
try:
    socket.setdefaulttimeout(2) sock =
    socket.socket(socket.AF_INET, socket.SOCK_DGRAM) sock.sendto(b'\x00',
    (ip, 53)) data,
    _ = sock.recvfrom(512)
    return True
except Exception:
    return False

# Function to check recursion on a DNS server def check_recursion(ip):

try:
    query = dns.message.make_query("www.example.com", dns.rdatatype.A, want_dnssec=True) response =
    dns.query.udp(query, ip, timeout=2)
```

```

        if response.flags & dns.flags.RA: # RA (Recursion Available) flag
            return True
        return False
    except Exception:
        return False

# Function to perform the search in Shodan and update the interface def
perform_search(api_key, output_text):
    log_message(output_text, "[+] Starting scan in Shodan...") try: api = shodan.

    Shodan(api_key) query = 'port:53'
    results = api.
    search(query, page=1) # Can be modified to go through several pages log_message(output_text, f"[+]
    {len(results['matches'])} servers found.")

    for service in results['matches']: ip =
        service['ip_str'] country
        = service.get('location', {}).get('country_name', 'Unknown') lat = service.get('location',
        {}).get('latitude', None) lon = service.get('location',
        {}).get('longitude', None) software = service.get('product',
        'Unknown')

        log_message(output_text, f"[-] Checking IP: {ip} ...") dns_active =
        check_dns(ip) recursive =
        check_recursion(ip) secure = any(ip ==
        dns_ip for dns_ip in DNS_SEGUROS.values())

        if dns_active:
            location = get_location(lat, lon) if lat and lon else "Coordinates not available" log_message(output_text,
            f"[y] Active DNS: {ip}\nCountry: {country}\nLocation: {location}\nSoftware: {software}\nRecursive: {'Yes' if recursive else 'No'}\nSecure: {'Yes' if secure else
            'No'}\n")
        else:
            log_message(texto_salida, f"[y] Sin reply: {ip}") except
    shodan.APIError as e:
        log_message(texto_salida, f"[!] Error de Shodan: {str(e)}")
        messagebox.showerror("Error de Shodan", str(e))
    except Exception as e:
        log_message(texto_salida, f"[!] Error inesperado: {str(e)}")
        messagebox.showerror("Error", str(e))

# Function to start the search in a separate thread def
start_search(api_key, output_text):
    if not api_key:
        messagebox.showwarning("Warning", "Please enter a Shodan API key.")
        return

    output_text.delete('1.0', tk.END) thread
    = Thread(target=perform_search, args=(api_key, output_text)) thread.start()

# Configure the graphical interface def
create_interface():
    window = tk.Tk()
    window.title("DNS Scanning")
    window.geometry("600x500")

    api_label = tk.Label(window, text="Shodan API Key:") api_label.pack(pady=5)

    entry_api = tk.Entry(window, width=50, show="")
    entry_api.pack(pady=5)

    texto_salida = scrolledtext.ScrolledText(ventana, width=70, height=25)
    texto_salida.pack(pady=10)

    start_button = tk.Button(window, text="Start Scan", command=lambda: start_search(api_input.get(), output_text)) start_button.pack(pady=5)

```

```
        window.mainloop()

if __name__ == "__main__":
    crear_interfaz()
```

Annex 4:

```
from scapy.all import *
```



```
def verify_advanced_dns(ip):

    try:

        # Make a standard DNS query

        consulta = IP(dst=ip) / UDP(dport=53) / DNS(rd=1, qd=DNSQR(qname="example.com"))

        response = sr1(query, timeout=2, verbose=False)

        # Check if we received a valid response

        if response and response.haslayer(DNS) and response.getlayer(DNS).ancount > 0:

            return True

        else:

            return False

    except Exception as e:

        print(f"Error checking DNS: {e}")

        return False
```

Annex 5:

```
import tkinter as tk

from tkinter import scrolledtext, messagebox

import shodan

import socket

from geopy.geocoders import Nominatim

from threading import Thread

import dns.query

import dns.message

import dns.query

import dns.message

# List of secure DNS for comparison

DNS_SEGUROS = {"Quad9": "9.9.9.9", "Cloudflare": "1.1.1.1", "Google": "8.8.8.8"}

# Function to print messages in the graphical interface
```

```
def log_message(output_text, message):

    output_text.insert(tk.END, message + "\n")

    output_text.see(tk.END)

    output_text.update()


# Function to get the location based on coordinates

def get_location(lat, lon):

    try:

        geolocator = Name(user_agent="geoapiExercises")

        location = geolocator.reverse(f"{lat}, {lon}", language='es')

        return location.address if location else "Unknown location"

    except Exception as e:

        return f"Error getting location: {str(e)}"


# Function to check if a DNS server responds - THIS PART OF CODE IS MODIFIED


def verify_dns(ip):

    try:

        # Make a standard DNS query to a DNS server

        query = dns.message.make_query("www.example.com", dns.rdatatype.A)

        response = dns.query.udp(query, ip, timeout=2)

        # If a response is received and it has the response section, the DNS server is active

        if response and response.answer:

            return True

        else:

            return False

    except Exception as e:

        return False # If an error occurs (timeout, etc.), the server is considered unresponsive


# Function to check recursion on a DNS server

def verify_recursion(ip):

    try:

        query = dns.message.make_query("www.example.com", dns.rdatatype.A, want_dnssec=True)

        response = dns.query.udp(query, ip, timeout=2)

        if response.flags & dns.flags.RA: # RA (Recursion Available) flag

            return True
```

```
        return False

    except Exception:

        return False

# Function to search Shodan and update the interface

def perform_search(api_key, output_text):

    log_message(output_text, "[+] Starting scan on Shodan...")

    try:

        api = shodan.Shodan(api_key)

        query = 'port:53'

        results = api.search(query, page=1) # Can be modified to traverse multiple pages

        log_message(output_text, f"[+] {len(results['matches'])} servers found.")

        for service in results['matches']:

            ip = service['ip_str']

            country = service.get('location', {}).get('country_name', 'Unknown')

            lat = servicio.get('location', {}).get('latitude', None)

            lon = servicio.get('location', {}).get('longitude', None)

            software = service.get('product', 'Unknown')

            log_message(texto_salida, f"[-] Verificando IP: {ip} ...")

            dns_active = verify_dns(ip)

            recursive = check_recursion(ip)

            seguro = any(ip == dns_ip for dns_ip in DNS_SEGUROS.values())

            if dns_active:

                location = get_location(lat, lon) if lat and lon else "Coordinates not available"

                log_message(output_text, f"[y] Active DNS: {ip}\nCountry: {country}\nLocation: {location}\nSoftware: {software}\nRecursive: {'Yes' if recursive else 'No'}\nSecure: {'Yes' if secure else 'No'}\n")

            else:

                log_message(output_text, f"[y] No response: {ip}")

    except shodan.APIError as e:

        log_message(texto_salida, f"[!] Error de Shodan: {str(e)}")

        messagebox.showerror("Error de Shodan", str(e))

    except Exception as e:

        log_message(output_text, f"[!] Unexpected error: {str(e)}")

        messagebox.showerror("Error", str(e))
```

```
# Function to start the search in a separate thread

def start_search(api_key, output_text):

    if not api_key:

        messagebox.showwarning("Warning", "Please enter a Shodan API key.")

        return

    output_text.delete('1.0', tk.END)

    thread = Thread(target=perform_search, args=(api_key, output_text))

    hilo.start()


# Configuring the graphical interface

def create_interface():

    window = tk.Tk()

    ventana.title("Escaneo DNS")

    ventana.geometry("600x500")

    api_label = tk.Label(window, text="Shodan API Key:")

    label_api.pack(pady=5)

    entrada_api = tk.Entry(ventana, width=50, show=*)

    entrada_api.pack(pady=5)

    texto_salida = scrolledtext.ScrolledText(ventana, width=70, height=25)

    output_text.pack(pady=10)

    start_button = tk.Button(window, text="Start Scan", command=lambda: start_search(api_input.get(), output_text))

    boton_initialize.pack(pady=5)

    window.mainloop()

if __name__ == "__main__":

    create_interface()
```

Annex 6:

```
import tkinter as tk

from tkinter import scrolledtext, messagebox

import shodan
```

```
import socket

from geopy.geocoders import Nominatim

from threading import Thread

import dns.query

import dns.message

import csv


# List of secure DNS for comparison

DNS_SEGUROS = {"Quad9": "9.9.9.9", "Cloudflare": "1.1.1.1", "Google": "8.8.8.8"}


def log_message(output_text, message):

    output_text.insert(tk.END, message + "\n")

    output_text.see(tk.END)

    output_text.update()


def get_location(lat, lon):

    try:

        geolocator = Name(user_agent="geoapiExercises")

        location = geolocator.reverse(f"{lat}, {lon}", language='es')

        return location.address if location else "Unknown location"

    except Exception as e:

        return f"Error getting location: {str(e)}"


def verify_dns(ip):

    try:

        query = dns.message.make_query("www.example.com", dns.rdatatype.A)

        response = dns.query.udp(query, ip, timeout=2)

        return bool(response and response.answer)

    except Exception:

        return False


def verify_recursion(ip):

    try:

        query = dns.message.make_query("www.example.com", dns.rdatatype.A, want_dnssec=True)

        response = dns.query.udp(query, ip, timeout=2)

        return bool(response.flags & dns.flags.RA)

    except Exception:
```

```
        return False

def generate_csv(results):

    try:

        results.sort(key=lambda x: (not x['Recursive'], not x['Safe']))

        with open("dns_scan_results.csv", "w", newline="", encoding='utf-8') as archivo:

            writer = csv.DictWriter(file, fieldnames=["IP", "Country", "Location", "Software", "Recursive", "Insurance"])

            escritor.writeheader()

            writer.writerows(results)

        return "dns_scan_results.csv"

    except Exception as e:

        return f"Error generating CSV: {str(e)}"

def perform_search(api_key, output_text):

    log_message(output_text, "[+] Starting scan on Shodan...")

    try:

        api = shodan.Shodan(api_key)

        query = 'port:53'

        results = api.search(query, page=1)

        log_message(output_text, f"[+] {len(results['matches'])} servers found.")

        dns_results = []

        for service in results['matches']:

            ip = service['ip_str']

            country = service.get('location', {}).get('country_name', 'Unknown')

            lat = servicio.get('location', {}).get('latitude', None)

            lon = servicio.get('location', {}).get('longitude', None)

            software = service.get('product', 'Unknown')

            log_message(texto_salida, f"[-] Verificando IP: {ip} ...")

            dns_active = verify_dns(ip)

            recursive = check_recursion(ip)

            seguro = any(ip == dns_ip for dns_ip in DNS_SEGUROS.values())

            if dns_active:

                location = get_location(lat, lon) if lat and lon else "Coordinates not available"

                dns_info = {"IP": ip, "Country": country, "Location": location, "Software": software, "Recursive": recursive, "Insurance": insurance}
```

```
        dns_results.append(dns_info)

        log_message(output_text, f"[y] Active DNS: {ip}\nCountry: {country}\nLocation: {location}\nSoftware: {software}\nRecursive: {'Yes' if recursive
else 'No'}\nSecure: {'Yes' if secure else 'No'}\n")

    else:

        log_message(output_text, f"[y] No response: {ip}")

    csv_file = generate_csv(dns_results)

    log_message(output_text, f"[+] Generated CSV file: {csv_file}")

except shodan.APIError as e:

    log_message(texto_salida, f"[!] Error de Shodan: {str(e)}")

    messagebox.showerror("Error de Shodan", str(e))

except Exception as e:

    log_message(output_text, f"[!] Unexpected error: {str(e)}")

    messagebox.showerror("Error", str(e))

def start_search(api_key, output_text):

    if not api_key:

        messagebox.showwarning("Warning", "Please enter a Shodan API key.")

        return

    output_text.delete('1.0', tk.END)

    thread = Thread(target=perform_search, args=(api_key, output_text))

    hilo.start()

def create_interface():

    window = tk.Tk()

    ventana.title("Escaneo DNS")

    ventana.geometry("600x500")

    api_label = tk.Label(window, text="Shodan API Key:")

    label_api.pack(pady=5)

    entrada_api = tk.Entry(ventana, width=50, show="*")

    entrada_api.pack(pady=5)

    texto_salida = scrolledtext.ScrolledText(ventana, width=70, height=25)

    output_text.pack(pady=10)
```

```
start_button = tk.Button(window, text="Start Scan", command=lambda: start_search(api_input.get(), output_text))
```

```
boton_initialize.pack(pady=5)
```

```
window.mainloop()
```

```
if __name__ == "__main__":
```

```
    create_interface()
```